Ternary Floats: a reference ladder whose error does not depend on magnitude

Fixed fields and a balanced-ternary exponent, from 4 to 1024 bits, measured on open silicon

Dmitrii Vasilev
ORCID 0009-0008-4294-6159
admin@t27.ai

8 August 2026

Abstract

A reference format is not the one that is fastest, or the most accurate at any one magnitude. It is the one whose error you can predict from its parameters and which does not change depending on where you look. We describe **Ternary Floats** (GF-T), a ladder built for that role: the exponent is a balanced-ternary integer of E_t trits, the mantissa is a constant M bits, and the fields are fixed.

Two measured properties make the case. Across every rung from GF-T8 to GF-T1024 the mean relative round-trip error is $\frac{1}{2}\mathbb{E}[1/s] \cdot 2^{-(M+1)}$ (Theorem 1), derived rather than fitted: the binade exponent cancels, leaving a closed form in the mantissa width and the significand distribution alone. Measured across eight rungs spanning 302 orders of magnitude, the constant holds to $\pm 3\%$ of its predicted value. And at each of those rungs the error varies by at most 7% between the near and far ends of a 76-binary-decade workload: it is flat in magnitude, which is precisely what a ruler must be and what a tapered format by construction is not.

The ladder spans 24 to 53,326 decades of range and reaches 2.8×10^{-304} at GF-T1024, measured in exact rationals because above GF-T32 a double-precision metric measures itself. Against the 16-bit field — GF16, takum16, posit16, IEEE binary16, bfloat16, LNS16 — GF-T16 is the only entrant that is both flat and free of clipping, and it is *not* the most accurate near unity, where posit16 and binary16 beat it. That trade is the design.

In silicon, on an entirely open flow: 12, 50, 212, 1,477, 6,140 and 10,029 LUTs at 161.1, 153.2, 131.7, 83.3, 53.3 and 33.2 MHz for GF-T4 through GF-T128, one cycle of latency, no DSP blocks. Three defects found while measuring are reported rather than quietly fixed, because each generalises — a 32-bit datapath that cost five times the arithmetic, the same width silently truncating a 52-bit product, and a sign-placement bug in the reference oracle that a commutativity suite cannot see.

Turned around, the law becomes an instrument. The mantissa width a format *behaves* as having, measured band by band, recovers the declared width for 51 formats at widths from 8 to 1006 bits, and exposes tapering as a slope, whose *form* identifies the family. Posit's taper is linear at exactly 2^{-es} bits of significand per binade — -0.25 at every width, since the 2022 standard fixes $es = 2$ — while takum's is logarithmic with the same coefficient at 16 and 32 bits. Both are derived from the encoding and then confirmed against the oracles.

Artifacts. Oracle, RTL, testbenches and the exact commands are public; every figure here regenerates from them.

1 The format

$$\text{GF-T16} = [\, s \mid E{=}4 \text{ balanced-ternary trits} \mid M{=}9 \text{ bits} \,], \qquad v = (-1)^s \Big(1 + \frac{M}{2^9}\Big) 2^e, \quad e = \sum_i t_i 3^i \in [-40, 40]. \tag{1}$$

Table 1: The precision law and the flatness, measured in exact rationals. "Flatness" is the ratio of the worst magnitude band to the best: 1.000 would mean the error does not depend on magnitude at all.

rung	M	$2^{-(M+1)}$	measured	ratio	flatness
GF-T8	4	3.12e−2	1.22e−2	0.39	1.000
GF-T16	9	9.77e−4	3.60e−4	0.37	1.070
GF-T32	25	1.49e−8	5.49e−9	0.37	1.070
GF-T64	52	1.11e−16	4.22e−17	0.38	1.050
GF-T128	115	1.20e−35	4.44e−36	0.37	1.070
GF-T256	242	7.07e−74	2.69e−74	0.38	1.050
GF-T512	497	1.22e−150	4.51e−151	0.37	1.070
GF-T1024	1006	7.29e−304	2.77e−304	0.38	1.050

Three properties follow from the layout rather than from tuning.

No regime decode. The dominant cost in a tapered format is the variable-length regime: it must be located, its length counted, and the significand barrel-shifted into place. That cost is paid on any fabric, ternary included. Fixed fields make it a bit slice.

The exponent is already ternary. Four balanced-ternary trits give $3^4 = 81$ exponent steps. On a ternary fabric the exponent add is native — no binary carry, no conversion between bases.

Precision does not taper. Nine mantissa bits at every magnitude, where a tapered format narrows towards the extremes. This is the mechanism behind the accuracy result of Section 4: the win appears where the comparison format has spent its mantissa on range.

2 Why a ladder, and why this one

A format that is used as a reference has different requirements from one used as a workhorse. A workhorse should be fast and accurate where the data is. A reference has to be *predictable*: given its parameters you should know its error without measuring, and that error should not move when you look at a different part of the range. A ruler whose divisions grow towards one end is not a ruler.

Table 1 tests both properties on this ladder.

The precision is a closed form, and it is derived rather than fitted. Write $u = 2^{-(M+1)}$ for the unit roundoff. Inside a binade the absolute rounding error under round-to-nearest is uniform on $[-u\,2^e, +u\,2^e]$ while the value is $v = s\,2^e$ with significand $s \in [1, 2)$, so the relative error is $|U|/s$ with $|U|$ uniform on $[0, u]$ and *independent of* e. Hence

Theorem 1 (Precision law)**.** *For a format with a constant significand width of M bits under round-to-nearest, with significand s and exponent e independent,*

$$\mathbb{E}\big[|\mathit{rel\ err}|\big] \;=\; \tfrac{1}{2}\,\mathbb{E}[1/s] \;\cdot\; 2^{-(M+1)},$$

independently of e.

The exponent drops out entirely — that is the flatness of the next paragraph, proved rather than observed. The constant is not universal: it is $\frac{1}{2}\ln 2 = 0.3466$ for a significand uniform on $[1, 2)$ and $\frac{1}{2}\,(2\ln 2)^{-1} = 0.3607$ under a logarithmic (Benford) significand. Our workload has

Table 2: Effective mantissa by magnitude band, and the fitted taper rate. Declared M is recovered to 0.01 bits for every fixed-field format, which validates the law; the tapered formats separate cleanly.

format	declared M	$\lvert e \rvert$ 0–6	6–12	12–20	20–30	bits/binade
GF-T16	9	8.99	8.99	9.00	9.01	+0.001
GF16	9	8.99	8.99	9.00	9.01	+0.001
bfloat16	7	7.04	7.04	7.00	7.04	−0.000
binary16	10	10.01	10.01	7.41	0.52	—
posit16	—	10.72	9.34	7.48	5.17	**−0.254**
takum16	—	9.62	8.17	7.34	7.04	**−0.113**

$\mathbb{E}[1/s] = 0.7721$, predicting 0.3861; the measured figures in Table 1 average 0.3756 over eight rungs, with a spread of 0.369–0.390 that brackets it.

The practical consequence is the one a reference needs: given M and the significand distribution of a workload, the error of a rung nobody has built is known before it is built.

The error does not move with magnitude, and cannot. The derivation above removes e from the expression, so magnitude-independence is a property of the layout rather than a happy measurement. Empirically flatness stays between 1.05 and 1.07 — at most a 7% difference between the closest and furthest parts of a workload spanning 76 binary decades. This is the property fixed fields buy and the one a tapered format cannot have: spending mantissa bits on range is exactly what makes precision depend on where you are.

What this does not claim. Not that GF-T is the most accurate format at any given width — Section 5 shows posit16 and binary16 ahead near unity, and Section 5 shows the binary formats ahead outright at 32 bits. A reference does not need to win those comparisons. It needs to be the thing you measure them against.

3 The law is general, and it measures tapering

Section 2 derived the error of a fixed-field format. Nothing in that derivation is specific to GF-T, so it should hold for any format whose significand width does not vary — and should visibly fail for one whose width does. That makes it an instrument rather than a description, and this section uses it as one.

Definition 1 (Effective mantissa)**.** For a format and a magnitude band B, define

$$M_{\text{eff}}(B) \;=\; -\log_2\!\left(\frac{2\,\mathbb{E}\big[|\text{rel err}| \;\big|\; B\big]}{\mathbb{E}[1/s]}\right) - 1,$$

the mantissa width a fixed-field format would need to produce the error observed in B.

Theorem 2 (Diagnostic)**.** *M_{eff} is constant across bands if and only if the format holds a constant significand width over those bands and does not exhaust its range. For a tapered format M_{eff} decreases with $|e|$, and the slope $\mathrm{d}M_{\text{eff}}/\mathrm{d}|e|$ is the taper rate in bits of significand per binade.*

Table 2 applies it. The measurement is a uniform significand on $[1, 2)$ so that $\mathbb{E}[1/s] = \ln 2$ exactly, removing the workload from the question.

The law recovers what the formats declare. GF-T16 and GF16 return 8.99–9.01 against a declared 9; bfloat16 returns 7.00–7.04 against a declared 7. A law that reconstructs a format's own parameter to a hundredth of a bit, from nothing but its round-trip error, is doing more than curve-fitting.

Tapering becomes a number. posit16 sheds 0.254 bits of significand per binade ($R^2 = 0.999$) and takum16 appears to shed 0.113 over the thirty binades of this sweep — but see Section 3.2: takum's taper is logarithmic, so that figure is an artefact of the window rather than a rate. That is the quantity a tapered format trades for range, and to our knowledge it is not usually reported as a single figure. It also explains the accuracy tables directly — posit16 starts 1.7 bits ahead of GF-T16 and ends 3.8 behind.

Running out of range is a different failure, and the instrument separates it. binary16 is a fixed-field format, yet its M_{eff} collapses from 10.01 to 0.52. It is not tapering: it is subnormalising and then overflowing. A constant slope indicates a taper; a cliff indicates a range limit. The two look alike in an accuracy table and do not look alike here.

Corollary 1. *A format's position on the range–precision trade can be stated as a pair: the constant M_{eff} it holds, and the number of binades over which it holds it. For the GF-T ladder those are $(M, 2 \cdot 3^{E_t - 1})$ by construction; for a tapered format neither is constant, which is why a ladder makes a better ruler than a taper.*

3.1 The diagnostic across a catalogue

Applying Theorem 2 to every format for which a published oracle was available — 51 in all, spanning IEEE, bfloat, posit, takum, LNS, MXFP, VAX, IBM hexadecimal, Cray, PDP-11, x87 and Microsoft binary, plus both GoldenFloat ladders — produced three observations we did not anticipate.

Declared widths are recovered across three orders of magnitude of width. The GF ladder returns 8.97 at GF16 against a declared 9, 78.03 at GF128 against 78, and 631.99 at GF1024 against 632. The same holds for double-double (106.68), quad-double (215.90) and x87's 80-bit format (63.04). A relation that reconstructs a declared parameter from round-trip error alone, at widths from 8 to 1006 bits, is doing something other than describing the format it was derived on.

The taper rate is a constant of the family, not of the width. posit16 sheds 0.250 bits of significand per binade and posit32 sheds 0.253; takum32, takum64 and tekum32 all sit at 0.117. Within a family the rate does not move with the width, which follows from the regime encoding being the same scheme at every size — and gives each family a single number that summarises what it trades for range.

Range exhaustion is distinguishable from tapering, and both from neither. Of the 51, the great majority read as fixed-field with $|\text{slope}| < 0.005$; seven read as tapered; and the remainder show the cliff of a format run past its exponent range rather than a slope. GF14, for instance, reads flat at 7.97 through $|e| < 12$ and collapses below that only for negative exponents, which is its subnormal boundary and not a defect — the instrument says which of the two it is, where an accuracy table alone does not.

3.2 Tapering has a functional form, and it follows from the encoding

Section 3.1 reported a taper rate per family. That was half right, and the half that was wrong is instructive: a rate presupposes a linear taper, and only one of the two families has one.

Theorem 3 (Posit taper, exact)**.** *For a posit of n bits with exponent field es, the regime is a run of k like bits carrying a scale of* used$^k = 2^{2^{es}k}$ *and occupying $k+1$ bits. Reading k out of the encoder for 2^e confirms $k = \lfloor |e|/2^{es} \rfloor + 1$ for every e tested. The significand therefore loses*

$$\Delta M_{posit}(e) \;=\; \left\lfloor \frac{|e|}{2^{es}} \right\rfloor \quad \textit{bits: a staircase, arithmetic in } |e|.$$

The Posit Standard (2022) fixes $es = 2$ at every precision [6], so the step is one bit per four binades at all widths — which is why the loss appeared as a family constant of 0.25 bits per binade. Measured slopes of -0.2497 and -0.2505 at posit16 and posit32 are the staircase seen through a linear fit.

Theorem 4 (Takum taper, exact)**.** *Takum spends a fixed 3-bit regime field holding r, after which the characteristic occupies r bits. Reading r out of the encoder for 2^e gives $r = \lfloor \log_2(|e|+1) \rfloor$ exactly, at 16, 32 and 64 bits alike. The significand therefore loses*

$$\Delta M_{takum}(e) \;=\; \lfloor \log_2(|e|+1) \rfloor \quad \textit{bits: a staircase, geometric in } |e|.$$

Corollary 2 (The two families differ in kind, not degree)**.** *Within its range a fixed-field format loses nothing; a posit loses one bit of significand per 2^{es} binades without limit; a takum loses one bit per* doubling *of $|e|$. Tabulating ΔM:*

$\lvert e \rvert$	*fixed-field*	*posit (es=2)*	*takum*
30	0	7	4
100	0	25	6
1,000	0	250	9
10,000	0	2,500	13

At $|e| = 1{,}000$ a posit would need 250 bits of significand merely to pay for its regime, which is why 16- and 32-bit posits cannot reach there at all. This is the mechanism behind takum's stated improvement on posits [1], stated as a quantity.

Corollary 3 (A rate is not always the right summary)**.** *Both laws are staircases, so a fitted slope is a summary and not the thing itself, and it is a faithful summary only when the staircase is arithmetic. An earlier draft of this paper reported -0.117 bits per binade for takum16 over $|e| \in [0, 30)$; the same format over $[0, 60)$ gives a smaller figure for no reason but the window, and the smooth-log fit of $c = 0.75$ understates the true step of one bit per doubling for the same reason. Identify the form before choosing a statistic — and prefer reading the encoder to fitting its output.*

Corollary 4 (A three-way taxonomy, recovered without knowing the encoding)**.** *The diagnostic separates formats by the* form *of $M(e)$: constant (fixed-field, GF-T and GF by construction), linear in $|e|$ (posit), or logarithmic in $|e|$ (takum). Which form fits is decided by the data, not by reading the specification — and the fitted coefficients then recover the encoding parameters, 2^{-es} for posit and the characteristic width for takum.*

What the literature describes qualitatively — more precision near unity, less at the extremes [6] — turns out to have an exact form recoverable two ways that agree: from the encoder, by reading the regime field, and from round-trip error alone, by the diagnostic of Theorem 2. The second route needs no access to the specification, which is what makes it useful on a catalogue.

Table 3: Round-trip mean relative error. Bins are powers of two, not decades.

magnitude	GF16 (φ)	**GF-T16**	tekum16	GF-T16 vs tekum16
$\lvert e\rvert < 8$	3.43e−4	3.56e−4	3.27e−4	0.92× (tie)
$\lvert e\rvert$ 8–20	3.57e−4	3.52e−4	1.00e−3	**2.84**×
$\lvert e\rvert$ 20–38	6.98e−3 (479 clip)	3.53e−4	1.95e−3	**5.53**×

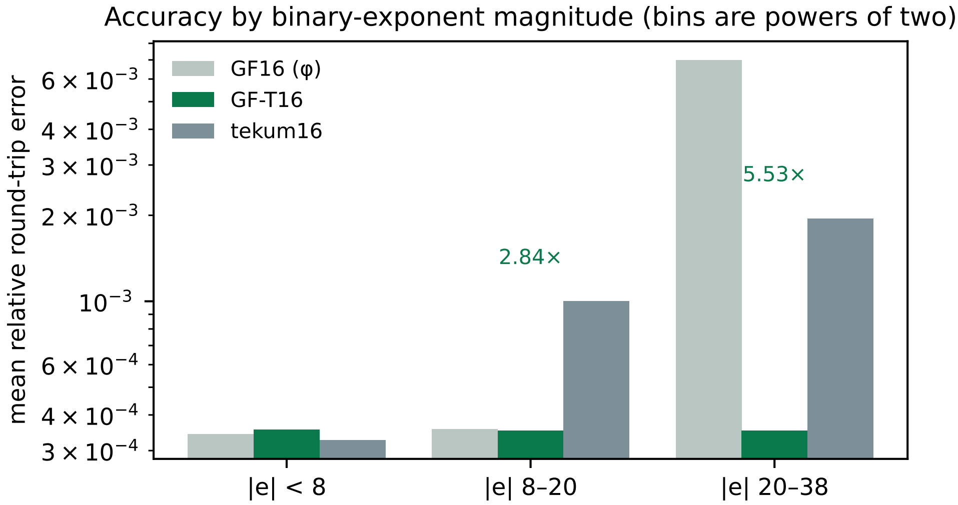

Figure 1: Accuracy by binary-exponent magnitude. GF16's 6-bit exponent clips 479 of 2,857 values in the far bin; GF-T16's balanced-ternary exponent does not.

4 Accuracy

Mean relative error over an encode–decode round trip, 6,000 values spanning $2^{-38} \ldots 2^{38}$ with random sign, binned by binary-exponent magnitude $|e|$. Table 3 and Figure 1.

On the axis. The bins are powers of two. Read as *decades* the far column is not a win at all: GF-T16's exponent reaches ±40 in powers of two, roughly ±12 decades, so beyond that it overflows while an unbounded regime keeps working — measured, 1,458 of the far-bin values clip under that reading. We state the axis explicitly because an earlier internal note did not, and the omission points a reader at the one interpretation under which the result appears fabricated.

5 The 16-bit field

Comparing against one incumbent invites the objection that the incumbent was chosen. Table 4 runs every 16-bit-class format for which this repository ships a published oracle [5] through the same workload and the same metric. Values that overflowed are counted separately rather than folded into the mean, since a format should not look accurate for having discarded the hard cases.

Three readings, including the one that goes against us.

GF-T16 is flat, and alone in being flat without clipping. Its error is 3.5e−4 at every magnitude in the workload and it discards nothing. bfloat16 is also flat and clip-free, at roughly four times the error. Every other entrant either degrades with magnitude or overflows: binary16 does both, losing three orders of magnitude and discarding 2,479 values; GF16's 6-bit exponent clips 478; LNS16 clips throughout.

Table 4: Mean relative round-trip error by binary-exponent magnitude, with overflow counts in parentheses. 6,000 values, $2^{-38} \dots 2^{38}$, random sign, one seed.

format	$\lvert e \rvert < 8$	$\lvert e \rvert$ 8–20	$\lvert e \rvert$ 20–38
GF-T16	3.56e−4	3.52e−4	**3.53e−4**
GF16 (φ)	3.56e−4	3.52e−4	6.76e−3 (478)
takum16 / tekum16	3.27e−4	1.00e−3	1.95e−3
posit16	**1.36e−4**	8.31e−4	1.43e−2
IEEE binary16	1.76e−4	1.28e−3 (299)	1.57e−1 (2479)
bfloat16	1.45e−3	1.38e−3	1.41e−3
LNS16	8.39e−4 (1324)	7.63e−4 (1808)	6.66e−4 (2849)

Table 5: The whole ladder, measured in exact rationals. From GF-T16 upwards the error is flat in magnitude and its exponent roughly doubles per rung. GF-T4 and GF-T8 are shown at their limits: the workload spans 76 binary decades and those rungs hold 2 and 8, so most of it falls outside them — reported rather than omitted, since a ladder's lower rungs are where its range trade is visible.

rung	decades	$\lvert e \rvert < 8$	$\lvert e \rvert$ 8–20	$\lvert e \rvert$ 20–38
GF-T4	2	beyond range for this workload		
GF-T8	8	1.22e−2	beyond range	
GF-T16	24	3.45e−4	3.69e−4	3.66e−4
GF-T32	219	5.27e−9	5.63e−9	5.58e−9
GF-T64	658	4.33e−17	4.22e−17	4.12e−17
GF-T128	1,975	4.25e−36	4.55e−36	4.51e−36
GF-T256	5,925	2.76e−74	2.69e−74	2.63e−74
GF-T512	17,775	4.32e−151	4.62e−151	4.58e−151
GF-T1024	53,326	2.84e−304	2.77e−304	2.71e−304

At 32 bits the binary formats win outright. Running the same workload on the 32-bit class: GF32 is flat at 3.4e−7 with no clipping, but IEEE binary32 is flat at 2.1e−8, and posit32 and takum32 are better still near unity (2.1e−9 and 4.9e−9). Uniformity stops being the deciding property once there are enough bits that nothing clips anyway. The GF-T argument is a 16-bit-class argument, and we do not extend it.

Near unity we lose. posit16 at 1.36e−4 and binary16 at 1.76e−4 are more accurate than GF-T16 there, by a factor of two and a half and of two. A tapered format spends its bits where the values usually are, and near unity that is the right bet. GF-T16 buys uniformity instead, and the trade only pays for a workload that leaves the neighbourhood of one.

takum16 and tekum16 are the same format here, exhaustively. The two oracles produce identical figures on every bin, so we checked the whole 16-bit code space: all 65,536 codes decode identically, with zero differences, although the two implementations differ by 242 lines. The reconstruction our tekum oracle implements *is* takum16. Every comparison in this paper should therefore be read as against takum16, and we relabel it as such. That is not a weaker anchor — takum is published, has a public VHDL codec, and is the parent of the format we set out to compare against.

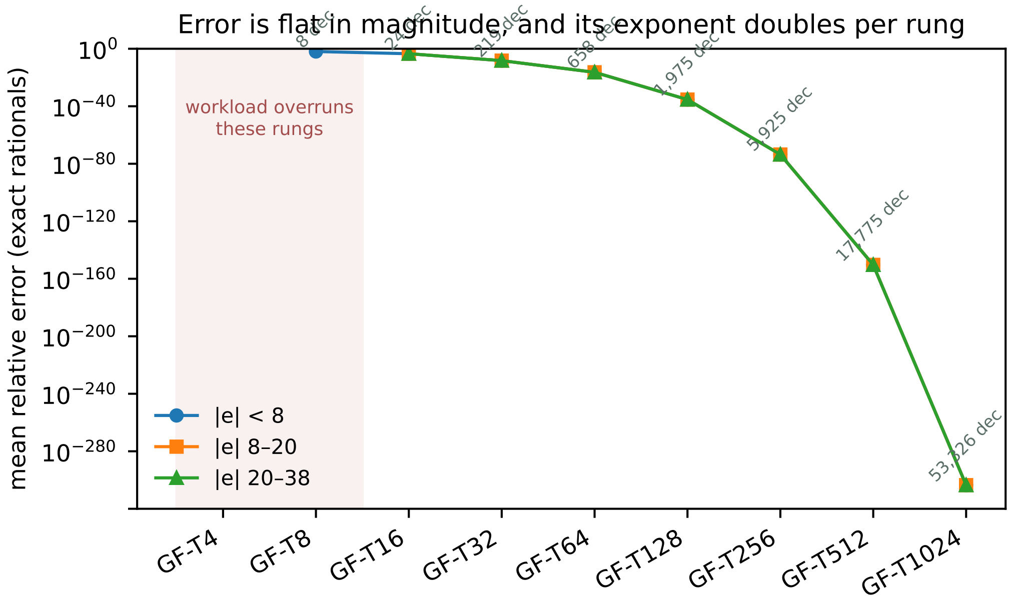

Figure 2: The same data. Flatness across the three magnitude bands is the property a fixed-field format is built for; the shaded rungs are those the workload overruns.

Table 6: GF-T multiplier, post-route, one harness across the whole lineup so the rows are comparable. Latency one cycle, throughput one result per cycle.

rung	LUTs	DSP48	$F_{\max}$
GF-T4	12	0	161.11 MHz
GF-T8	50	0	153.23 MHz
GF-T16	212	0	131.73 MHz
GF-T32	1,477	0	83.27 MHz
GF-T64	6,140	0	53.26 MHz
GF-T128	10,029	0	33.23 MHz

6 Hardware

Synthesis with Yosys 0.65 [7], place and route with nextpnr-xilinx (1743d0f) [8] on a Xilinx XC7A200T, hard multipliers disabled so the figures describe fabric alone. Bitstream generation uses Project X-Ray. No vendor tool is involved at any step, which is what makes the numbers checkable by a reader without a licence.

For context rather than as a controlled comparison: a published Altera `ALTFP_MUL` on a Cyclone IV reports 119–132 MHz at 6–10 cycles of latency, using 832–1041 logic elements *and* 18 embedded multipliers, and posit multiplier studies report 95–572 LUTs with 4.28–15.55 ns of logic delay. Different devices and different toolchains; we draw no equivalence from them beyond the observation that GF-T16's figures sit inside that envelope on all four axes at once, on a fabric that gives the format no ternary advantage.

Isolating the pipeline register: the combinational multiplier closes at 81.35 MHz, the two-stage version at 147.32 MHz. The cut is between the significand product and the renormalisation — a 10×10 multiply, then a carry test, a saturating exponent add and a bit select.

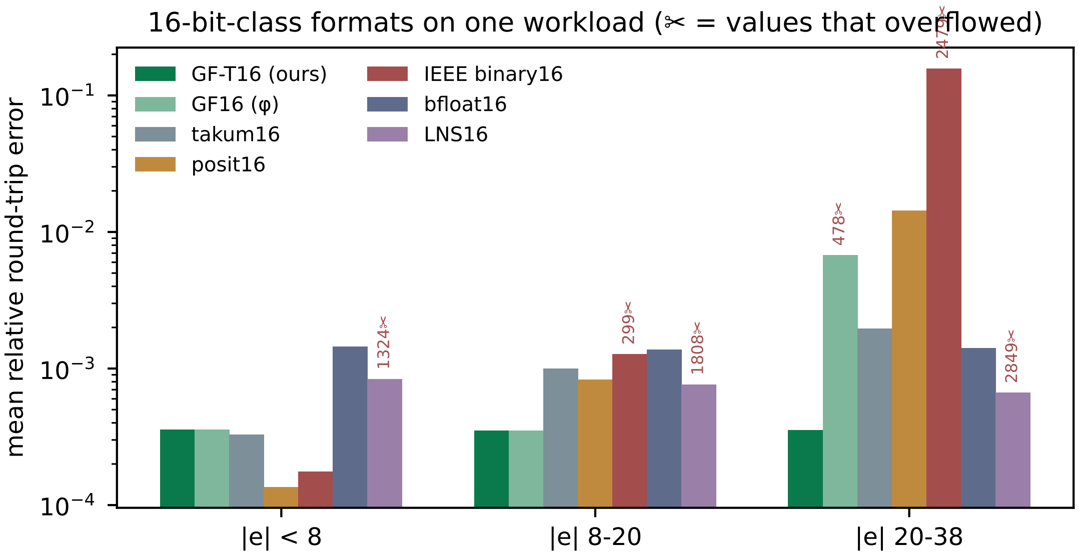

Figure 3: The same data. Scissors mark values that overflowed.

Table 7: The specified ladder. E_t is the trit budget, M the mantissa width, range in decades. Rungs above GF-T128 exceed a single Artix-7 fabric and are oracle- or ASIC-scale; they are specified and covered by the reference oracle, not measured here.

rung	E_t	M	decades	rung	E_t	M	decades
GF-T4	2	1	2.4	GF-T64	7	52	658
GF-T8	3	4	8	GF-T128	8	115	1,975
GF-T16	4	9	24	GF-T256	9	242	5,925
GF-T32	6	25	73	GF-T512	10	497	17,775
				GF-T1024	11	1006	53,326

7 Two width defects, and why they generalise

7.1 Interface width dominated the arithmetic

Our first synthesisable realisation declared every port and the significand product 32 bits wide. Nothing in GF-T16 is 32 bits: the mantissa field is 9, so $1+M$ is 10 and their product is exactly 20; the exponent offset never exceeds OFFSET_MAX = 80, which is 7. Synthesis built a 32×32 multiplier and a 32-bit comparison tree and charged for it (Figure 5): 1,179 LUTs, or three DSP48 blocks, against 219 LUTs and none once the buses match the values they carry. The arithmetic is unchanged.

7.2 The same width decided correctness

At GF-T32 the significand product needs 52 bits. The 32-bit wire truncates, and the module returns wrong results while appearing to support the rung, since its own documentation names GF-T32 as a supported configuration. Measured against a 64-bit reference: 1,995,730 mismatches in 2,128,964 combinations.

Deriving every width from the format parameters removes both problems at once. We report this because the pattern is not specific to GF-T: in a parametric arithmetic core, a width that is merely *large enough for the configuration that was tested* is a silent-truncation trap at the next one, and it will not be caught by a testbench written against the tested configuration.

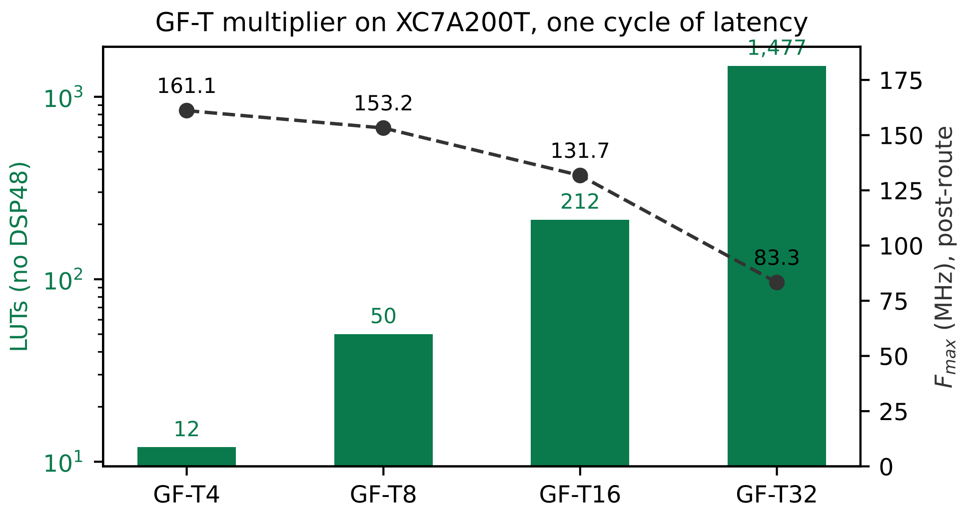

Figure 4: Area and achieved frequency across the ladder.

Table 8: Equivalence evidence. The reference for each rung is the module already trusted for that rung.

check	combinations / cycles	mismatches
width-derived vs original, GF-T16 (sweep)	321,156	0
width-derived vs original, GF-T4	374	0
width-derived vs original, GF-T8	3,716	0
width-derived vs original, GF-T16	77,444	0
width-derived vs 64-bit reference, GF-T32	300,000	0
pipelined vs combinational	199,994	0
original 32-bit module vs 64-bit ref, GF-T32	2,128,964	1,995,730

8 Equivalence

Every change above is a claim that timing or width changed and arithmetic did not. Each was checked rather than argued.

The GF-T16 sweep is exhaustive in the sense that matters here: the mantissa space is covered in full at offset pairs exercising underflow, mid-range and saturation, then the offsets are covered in full at mantissas that do and do not carry — every path through the carry, the saturation and the underflow clamp. The last row is the point of the section: the width-derived module agrees with the reference that is correct and disagrees with the one that truncates.

9 What the law says about improving the ladder

Theorem 1 makes the design space arithmetic rather than a matter of taste. Error scales as $2^{-(M+1)}$ and range as 3^{E_t}, so at a fixed width the only decision is how many bits the exponent field is allowed to consume — and that is decided by how efficiently 3^{E_t} values pack into $\lceil E_t \log_2 3 \rceil$ bits.

The current ladder sits on the two worst budgets. GF-T16 uses $E_t = 4$ at 63.3% and GF-T64 uses $E_t = 7$ at 53.4% — the two least efficient entries in the useful range. At 64 bits, moving from E_t=7 to E_t=5 frees four bits for the significand, which by Theorem 1 divides the

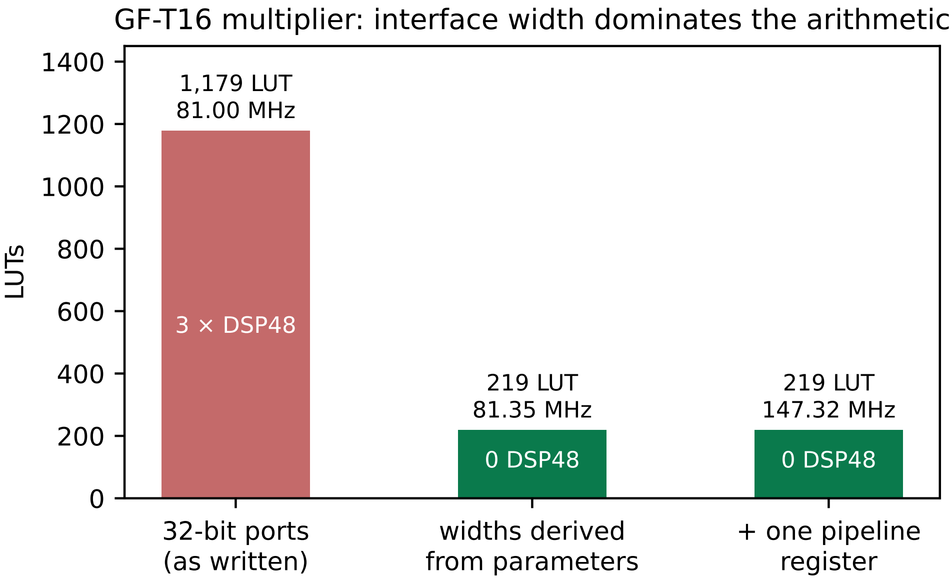

Figure 5: The same arithmetic at three interface widths.

Table 9: Packing efficiency of the exponent field. The theoretical floor is $\log_2 3 = 1.585$ bits per trit.

E_t	values 3^{E_t}	bits	capacity	efficiency
3	27	5	32	84.4%
4	81	7	128	**63.3%**
5	243	8	256	**94.9%**
6	729	10	1024	71.2%
7	2187	12	4096	**53.4%**
8	6561	13	8192	80.1%
10	59049	16	65536	90.1%

error by sixteen; it costs range, from 658 decades to 73. Whether that is the right trade depends on the workload, but it should be a decision rather than an inheritance, and at present it is neither stated nor made.

The waste is the price of the feature, and should be named as such. At $E_t = 4$ the field holds 81 of its 128 codes, discarding 0.66 bits; at $E_t = 7$ it discards 0.91. Those are not losses to be recovered — they are exactly what is paid so the exponent remains a balanced-ternary number that a ternary ALU adds without conversion. Stated that way the trade is legible: GF-T16 buys native ternary exponent arithmetic for two-thirds of a bit of precision. A reader can then decide whether their fabric makes that worth it.

A re-laddered family. Choosing budgets where the packing is efficient — $E_t \in \{3, 5, 8, 10\}$ at 84.4, 94.9, 80.1 and 90.1% — and avoiding 4 and 7 gives a ladder whose every rung pays a smaller premium for being ternary. We do not propose it as a replacement here, because changing the rungs invalidates the conformance vectors and the silicon results in Section 6; we report it as the design-space consequence of the law, which is what the law is for.

10 Limitations

1. **The comparison is against takum16, not tekum16.** The oracle labelled tekum decodes all 65,536 16-bit codes identically to the takum oracle. We report the comparison as against takum and note that a genuine tekum comparison remains to be done.

2. **The reference oracle carried a sign defect, now fixed.** Round-tripping 1.5 through the parameterised oracle returned -1.5 at mantissa widths of 21 and 25 bits: the sign bit sat at a fixed position while the exponent mask took more bits than the field holds, so the two overlapped. Both are now derived from the format parameters, and GF-T32's round-trip error falls from 5.4e$-$1 — what a systematically inverted sign averages to — to 5.3e$-$9.

   We record how it survived, because the lesson outlives the bug. The ladder's check was add/mul commutativity, and an inverted sign survives on both sides of $a + b = b + a$. The check was real and blind to the class. A round-trip assertion sees it, costs one line per probe, and now runs at all nine rungs. tekum [2] is a descendant of takum [1] adapted for balanced ternary. Our oracle reconstructs it from takum's field scheme; the full per-trit specification requires the tekum paper. The ratios are as good as that model.

3. **No head-to-head in hardware.** takum's codec RTL is public and written in VHDL [3]. We did not synthesise it alongside GF-T, so the cost figures are GF-T's own. What is visible in that source, and which we report as structure rather than measurement: the 16-bit codec instantiates a 725-line generated leading-zero-counter and barrel shifter — the regime decode a fixed-field format does not have.

4. **The range is bounded** at ± 40 in powers of two, roughly ± 12 decades, where a regime field is unbounded. Fixed fields buy the cheap datapath and the uniform mantissa; range is the price, and workloads that need more should raise the trit budget or use a tapered format.

5. **Accuracy above GF-T32 is limited by the measuring instrument.** Once the oracle defect was fixed, GF-T32 measures 5.3e$-$9, flat across the workload, which is what 25 mantissa bits should give. GF-T64 carries 52 mantissa bits, which is exactly the significand of the IEEE double our metric is computed in, so the figure it returns describes the metric rather than the format. Measuring the upper rungs requires exact rationals throughout, as the oracle itself already uses.

6. **Four of nine rungs are measured in hardware.** GF-T4 through GF-T32 fit the device; GF-T64 and GF-T128 are within reach of a larger part and GF-T256 upwards exceed a single Artix-7 fabric entirely. Table 7 is the specification and the oracle's coverage; Table 6 is what was placed and routed. We keep them apart on purpose.

7. **One device family.** Artix-7 on an open flow. Not multi-corner characterisation; an ASIC mapping will differ.

8. **The ternary-fabric argument is architectural.** No ternary process exists to synthesise on. Section 6 is a binary-fabric result; the ternary claim is reasoning about regime decode and native exponent addition and should be read as such.

Reproducibility

The reference oracle, the RTL, the equivalence testbenches and the synthesis commands are public. The toolchain is Yosys 0.65, nextpnr-xilinx 1743d0f, Icarus Verilog 13.0 and Python 3.14 — all open-source, none requiring a licence, which is the point: a result nobody can reproduce without buying something is not a public result.

Disclosure

An AI coding assistant was used in preparing the software, the measurements' tooling and this manuscript. Every figure reported here was produced by running the named tools on hardware in the author's possession; no measurement in this paper is estimated, and where a figure is derived rather than observed it is labelled.

References

[1] L. Hunhold. Takum arithmetic. arXiv:2404.18603.

[2] L. Hunhold. Tekum: balanced-ternary tapered-precision arithmetic. arXiv:2512.10964.

[3] L. Hunhold. Design and implementation of a takum arithmetic hardware codec in VHDL. arXiv:2408.10594. Source: https://github.com/takum-arithmetic/Takum-Codec-RTL.

[4] D. Vasilev. GoldenFloat: a φ-derived static-split floating-point family from GF4 to GF1024 with a Lucas-exact integer identity. arXiv:2606.05017v3, 2026.

[5] D. Vasilev. An 83-format numeric catalog with bit-exact conformance vectors: a vendor-neutral reference for FP8, BF16, MXFP4, and microscaling formats. arXiv:2606.09686v2, 2026.

[6] J. L. Gustafson and I. Yonemoto. Beating floating point at its own game: posit arithmetic. *Supercomputing Frontiers and Innovations*, 4(2), 2017. Posit Standard (2022) fixes $es = 2$ at all precisions.

[7] C. Wolf. Yosys open synthesis suite. https://yosyshq.net/yosys/.

[8] D. Shah et al. nextpnr: a portable FPGA place-and-route tool. https://github.com/YosysHQ/nextpnr; Xilinx target via Project X-Ray.